



# SI100B: Introduction to Information Science and Technology



## Lecture 4

# Announcement

---

## ▶ Homework 1

### ▶ Instructions & problems

▶ BB → 作业 (CS) → “Programming Assignment 1”

### ▶ Submission website: OJ

▶ See BB → 作业 (CS) → OJ使用说明

▶ 学校内网；校外访问需使用上科大VPN

### ▶ Due: 9pm, Oct. 9

▶ One more week than usual because of the holiday





# APPROXIMATION METHODS

# BETTER than GUESS-and-CHECK

---

- ▶ Last time: guess-and-check finds exact answer by enumerating a finite set of possible values (integers)
- ▶ In general
  - ▶ Set of possible answers can be infinite
  - ▶ Exhaustive enumeration can't test every possible value
  - ▶ Exact answer may not be accessible
- ▶ Need to find ways to get **approximation to an answer**
  - ▶ Our answer is “close enough” to ideal answer



# FINDING ROOTS

---

- ▶ Previously, we looked at using exhaustive enumeration/guess and check methods to find the **roots of perfect squares**
- ▶ Suppose we want to find the square root of any positive integer, or any positive number
- ▶ Question: What does it mean to find the square root of  $x$ ?
  - ▶ Find an  $r$  such that  $r*r = x$  ?
  - ▶ If  $x$  is not a perfect square, then not possible in general to find an exact  $r$  that satisfies this relationship; and **exhaustive search is infinite**



# APPROXIMATION

---

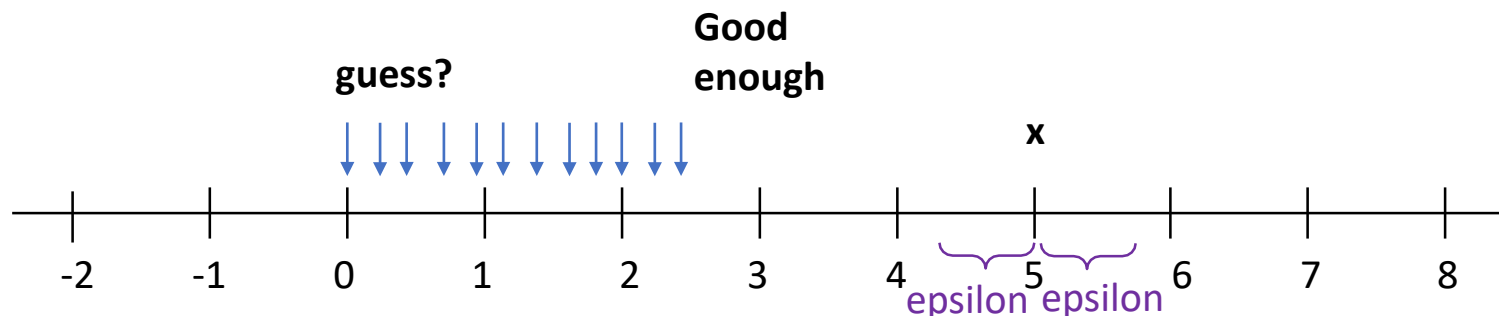
- ▶ Find an answer that is **“good enough”**
  - ▶ E.g., find  $r$  such that  $r*r$  is within a given (small) distance of  $x$
  - ▶ Use epsilon: given  $x$  we want to find  $r$  such that  $|r^2-x|<\epsilon$
- ▶ Algorithm
  - ▶ Start with guess **known to be too small** – call it  $g$
  - ▶ Increment by a small value (call it  $a$ ) to give a new guess  $g$
  - ▶ Check if  $g**2$  is close enough to  $x$  (within  $\epsilon$ )
  - ▶ Continue until get answer close enough to actual answer
- ▶ Looking at all possible **values  $g + k*a$**  for integer values of  **$k$** 
  - ▶ Similar to exhaustive enumeration
  - ▶ But cannot test all possibilities (infinite)



# APPROXIMATION ALGORITHM

---

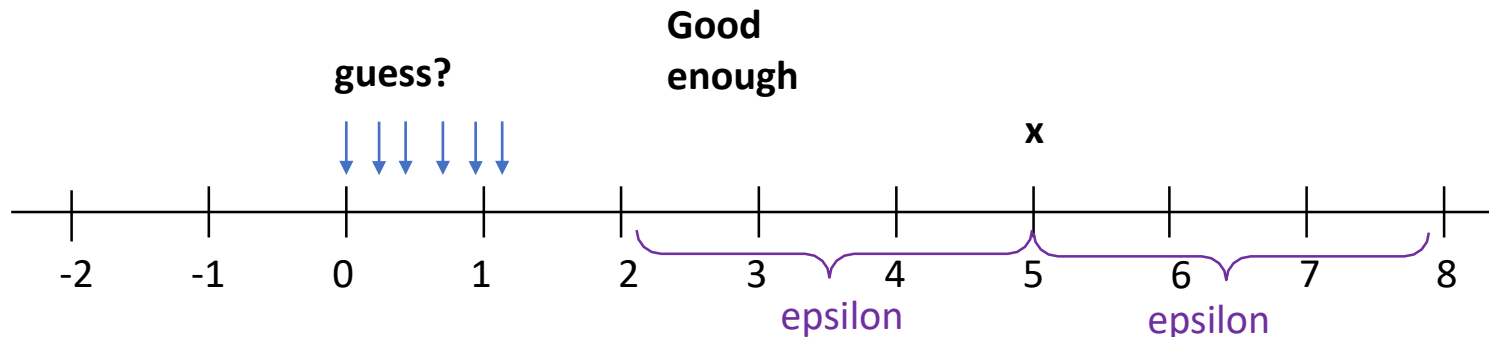
- ▶ In this case, we have **two parameters** to set
  - ▶ **epsilon** (how close are we to answer?)
  - ▶ **increment** (how much to increase our guess?)
- ▶ Performance will vary based on these values
  - ▶ In speed
  - ▶ In accuracy
- ▶ **Decreasing increment** size → slower program, but more likely to get good answer (and vice versa)



# APPROXIMATION ALGORITHM

---

- ▶ In this case, we have **two parameters** to set
  - ▶ **epsilon** (how close are we to answer?)
  - ▶ **increment** (how much to increase our guess?)
- ▶ Performance will vary based on these values
  - ▶ In speed
  - ▶ In accuracy
- ▶ **Increasing epsilon** → less accurate answer, but faster program (and vice versa)



# IMPLEMENTATION

---

```
x = 36
epsilon = 0.01
num_guesses = 0
guess = 0.0
increment = 0.0001

while abs(guess**2 - x) >= epsilon:
    guess += increment
    num_guesses += 1

print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```



# IMPLEMENTATION

---

```
x = 36
epsilon = 0.01
num_guesses = 0
guess = 0.0
increment = 0.0001
```

```
while abs(guess**2 - x) >= epsilon:
    guess += increment
    num_guesses += 1
```

*Will this loop always terminate?*

```
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```



# OBSERVATIONS with DIFFERENT VALUES for $x$

---

- ▶ For  $x = 36$ 
  - ▶ Didn't find 6
  - ▶ Took nearly 60,000 guesses
- ▶ Let's try:
  - ▶ 24
  - ▶ 2
  - ▶ 12345
  - ▶ 54321



```
x = 54321
epsilon = 0.01
numGuesses = 0
guess = 0.0
increment = 0.0001
```

```
while abs(guess**2 - x) >= epsilon:
    guess += increment
    numGuesses += 1
```

```
if numGuesses%100000 == 0:
    print('Current guess =', guess)
    print('Current guess**2 - x =', abs(guess*guess - x))
    input('continue?')
```

```
print('numGuesses =', numGuesses)
```

```
print(guess, 'is close to square root of', x)
```

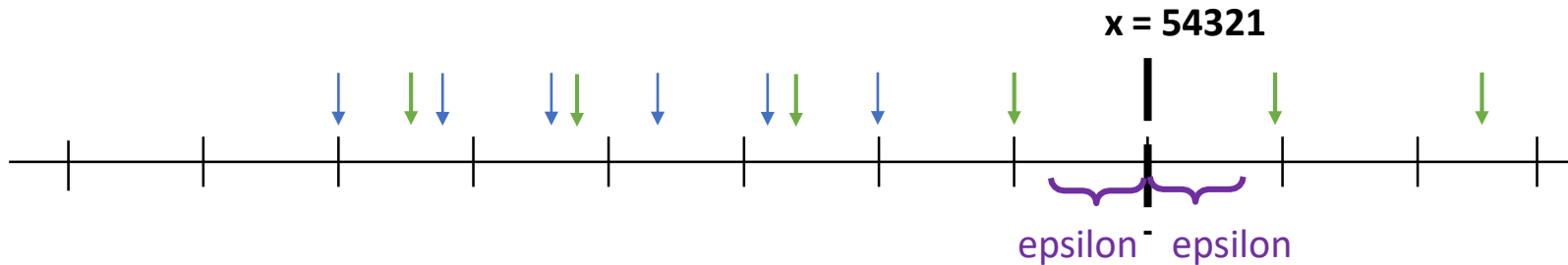
Debugging print statements  
every 100000 times through the  
loop, showing guess and how  
far away from epsilon we are



# WE OVERSHOT the EPSILON!

---

- ▶ Blue arrow is the guess
- ▶ Green arrow is  $\text{guess}^*2$



# SOME OBSERVATIONS

---

- ▶ Decrementing function eventually starts incrementing
  - ▶ So didn't exit loop as expected
- ▶ We have **over-shot the mark**
  - ▶ I.e., we jumped from a value too far away but too small to one too far away but too large
- ▶ We **didn't account for this possibility when writing the loop**
- ▶ Let's fix that



# LET'S FIX IT

---

```
x = 54321
epsilon = 0.01
numGuesses = 0
guess = 0.0
increment = 0.0001
while abs(guess**2 - x) >= epsilon and guess**2 <= x:
    guess += increment
    numGuesses += 1
print('numGuesses =', numGuesses)
if abs(guess**2 - x) >= epsilon:
    print('Failed on square root of', x)
else:
    print(guess, 'is close to square root of', x)
```



# LET'S FIX IT

---

```
x = 54321
epsilon = 0.01
numGuesses = 0
guess = 0.0
increment = 0.0001
while abs(guess**2 - x) >= epsilon and guess**2 <= x:
    guess += increment
    numGuesses += 1
print('numGuesses =', numGuesses)
if abs(guess**2 - x) >= epsilon:
    print('Failed on square root of', x)
else:
    print(guess, 'is close to square root of', x)
```

Same condition as guess-and-check,  
stop when you go past the last  
reasonable guess



# LET'S FIX IT

---

```
x = 54321
epsilon = 0.01
numGuesses = 0
guess = 0.0
increment = 0.0001
while abs(guess**2 - x) >= epsilon and guess**2 <= x:
    guess += increment
    numGuesses += 1
print('numGuesses =', numGuesses)
if abs(guess**2 - x) >= epsilon:
    print('Failed on square root of', x)
else:
    print(guess, 'is close to square root of', x)
```

Same condition as guess-and-check,  
stop when you go past the last  
reasonable guess

Exited b/c  $\text{guess}^2 > x$

Exited b/c  $\text{guess}^2$   
is within eps



# SOME OBSERVATIONS

---

- ▶ Now it stops, but **reports failure**, because it has over-shot the answer
- ▶ We can reset increment to 0.00001
  - ▶ Smaller increment means **more values will be checked**
  - ▶ Program will be slower



# LESSONS LEARNED in APPROXIMATION

---

- ▶ Can't use `==` to check an exit condition
- ▶ Need to be careful that looping mechanism doesn't **jump over exit test** and loop forever
- ▶ Need to think about **how close** an answer we want when **setting parameters** of algorithm
- ▶ **Tradeoff** exists between efficiency of algorithm and accuracy of result
- ▶ To get a good answer, this method can be painfully slow.
  - ▶ Is there a **faster way that still gets good answers**?
  - ▶ **YES!** We will see it later....





# BISECTION SEARCH

# CHANCE to WIN BIG BUCKS

---

- ▶ Suppose I attach a \$100 bill to a particular page in the text book, 448 pages long
  - ▶ If you can guess page in 8 or fewer guesses, you get big bucks
  - ▶ If you fail, you give me \$10
  - ▶ Would you want to play?
- Your chances are about 1 in 56*
- ▶ Now suppose on each guess I told you whether you were correct, or too low, or too high
  - ▶ Would you want to play in this case?

*Your chances are about 1 in 3*

---



# BISECTION SEARCH

---

- ▶ Apply it to problems with an **inherent order** to the **range** of possible answers
- ▶ Suppose we know answer lies within some interval
  - ▶ Guess **midpoint** of interval
  - ▶ If not the answer, check if **answer is greater than or less** than midpoint
  - ▶ **Change** interval
  - ▶ Repeat
- ▶ Process **cuts set of things to check in half** at each stage
  - ▶ Exhaustive search reduces them from  $N$  to  $N-1$  on each step
  - ▶ Bisection search reduces them from  $N$  to  $N/2$



# LOG GROWTH is BETTER

---

- ▶ **Algorithm comparison: guess-and-check vs. bisection search**
  - ▶ Checking answer one-by-one iteratively is linear in the number of possible guesses
  - ▶ Checking answer by guessing the halfway point is logarithmic on the number of possible guesses
    - ▶ Why? More on this later
  - ▶ Log algorithm is much **more efficient**

We will see discussion of  
relative costs of different  
algorithms in a few weeks



# BISECTION SEARCH for SQUARE ROOT

---

- ▶ Suppose we know that the **answer lies between 0 and  $x$**
- ▶ Rather than exhaustively trying things starting at 0, suppose instead we **pick a number in the middle** of this range



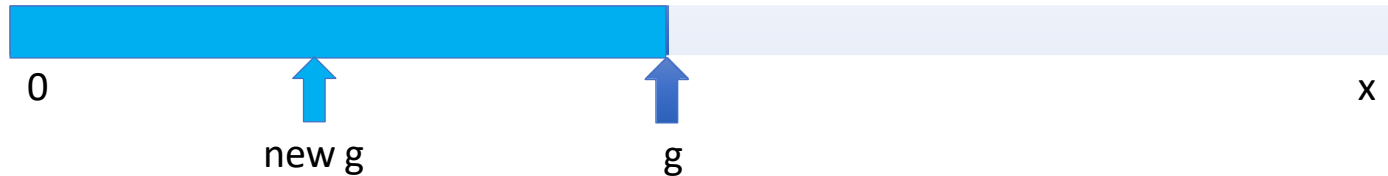
- ▶ If we are lucky, this answer is close enough



# BISECTION SEARCH for SQUARE ROOT

---

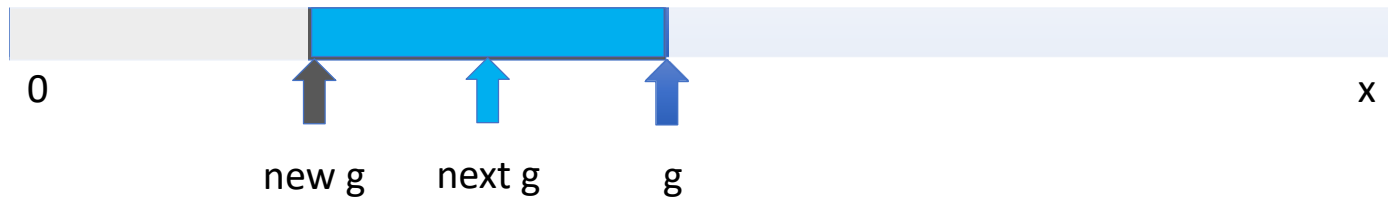
- ▶ If not close enough, **is guess too big or too small?**
- ▶ If  $g^2 > x$ , then know  $g$  is too big; so now search



# BISECTION SEARCH for SQUARE ROOT

---

- ▶ And if, for example, this new  $g$  is such that  $g^2 < x$ , then know  $g$  is too small; so now search



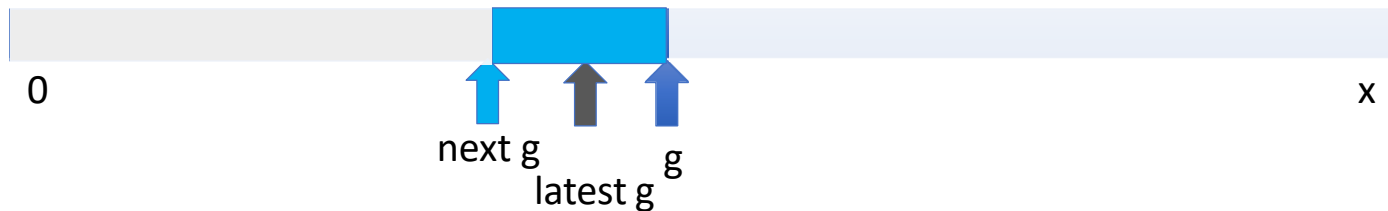
- ▶ At each stage, **reduce range of values to search by half**



# BISECTION SEARCH for SQUARE ROOT

---

- ▶ And if, for example, this next  $g$  is such that  $g^2 < x$ , then know  $g$  is too small; so now search



- ▶ At each stage, **reduce range of values to search by half**



# BIG IDEA

Bisection search takes advantage of properties of the problem.

- 1) The search space has an order
- 2) We can tell whether the guess was too low or too high



# YOU TRY IT!

---

- ▶ You are playing an EXTREME guessing game to guess a number EXACTLY. A friend has a decimal number between 0 and 10 (to any precision) in mind. The feedback on your guess is whether it is correct, too high, or too low. Can you use bisection search to quickly and correctly guess the number?



# SLOW SQUARE ROOT USING APPROXIMATION METHODS

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
guess = 0.0
increment = 0.00001
while abs(guess**2 - x) >= epsilon and guess**2 <= x:
    guess += increment
    num_guesses += 1
print('num_guesses =', num_guesses)
if abs(guess**2 - x) >= epsilon:
    print('Failed on square root of', x)
else:
    print(guess, 'is close to square root of', x)
```



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
```

*Initialize some stuff*

```
while abs(guess**2 - x) >= epsilon:
```

*What is repeated each time?*

```
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
```

```
low = 0
high = x
guess = (high + low)/2.0
```

*Initialize endpoints  
and guess*

```
while abs(guess**2 - x) >= epsilon:
```



*What is repeated each time?*

```
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
low = 0
high = x
guess = (high + low)/2.0
while abs(guess**2 - x) >= epsilon:
    if guess**2 < x :

    else:

    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

Check if guess squared is too low or too high compared to x



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
low = 0
high = x
guess = (high + low)/2.0
while abs(guess**2 - x) >= epsilon:
    if guess**2 < x :
        low = guess
    else:
        high = guess
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

*If guess was too low, reset the low  
endpoint to the guess*



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
low = 0
high = x
guess = (high + low)/2.0
while abs(guess**2 - x) >= epsilon:
    if guess**2 < x :
        low = guess
    else:
        high = guess
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

*If guess was too high, reset the high endpoint to the guess*



# FAST SQUARE ROOT

---

```
x = 54321
epsilon = 0.01
num_guesses = 0
low = 0
high = x
guess = (high + low)/2.0
while abs(guess**2 - x) >= epsilon:
    if guess**2 < x :
        low = guess
    else:
        high = guess
    guess = (high + low)/2.0
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

*Make a new guess using  
the new endpoints*



# LOG GROWTH is BETTER

---

- ▶ Brute force search for root of 54321 took over **23M guesses**
- ▶ With bisection search, reduced to **30 guesses!**
- ▶ The brute force method is **linear in size of problem**, because number of steps grows linearly as we increase problem size
- ▶ Bisection search is **logarithmic in size of problem**, because number of steps grows logarithmically with problem size
  - ▶ search space
  - ▶ first guess:  $N/2$
  - ▶ second guess:  $N/4$
  - ▶ kth guess:  $N/2^k$
  - ▶ when  $k = \log_2 N$ , we have one guess left to check
  - ▶ guess converges on the order of  $\log_2 N$  steps



# DOES IT ALWAYS WORK?

---

- ▶ Try running code for  $x$  such that  $0 < x < 1$
- ▶ We are **searching from 0 to  $x$**
- ▶ But **square root is greater than  $x$  and less than 1**
- ▶ Modify the code to choose the search space depending on value of  $x$



# BISECTION SEARCH – SQUARE ROOT for ALL x VALUES

---

```
x = 0.5  
epsilon = 0.01
```

```
if x >= 1:  
    low = 1.0  
    high = x  
else:  
    low = x  
    high = 1.0
```

```
guess = (high + low)/2
```

```
while abs(guess**2 - x) >= epsilon:  
    if guess**2 < x:  
        low = guess  
    else:  
        high = guess  
    guess = (high + low)/2.0
```

```
print(f'{str(guess)} is close to square root of {str(x)}')
```



## YOU TRY IT!

---

- ▶ Write code to do bisection search to find the cube root of positive cubes within some epsilon.



# NEWTON-RAPHSON

---

- ▶ General **approximation algorithm to find roots of a polynomial** in one variable

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

- ▶ Newton and Raphson showed that if  $g$  is an approximation to the root, then

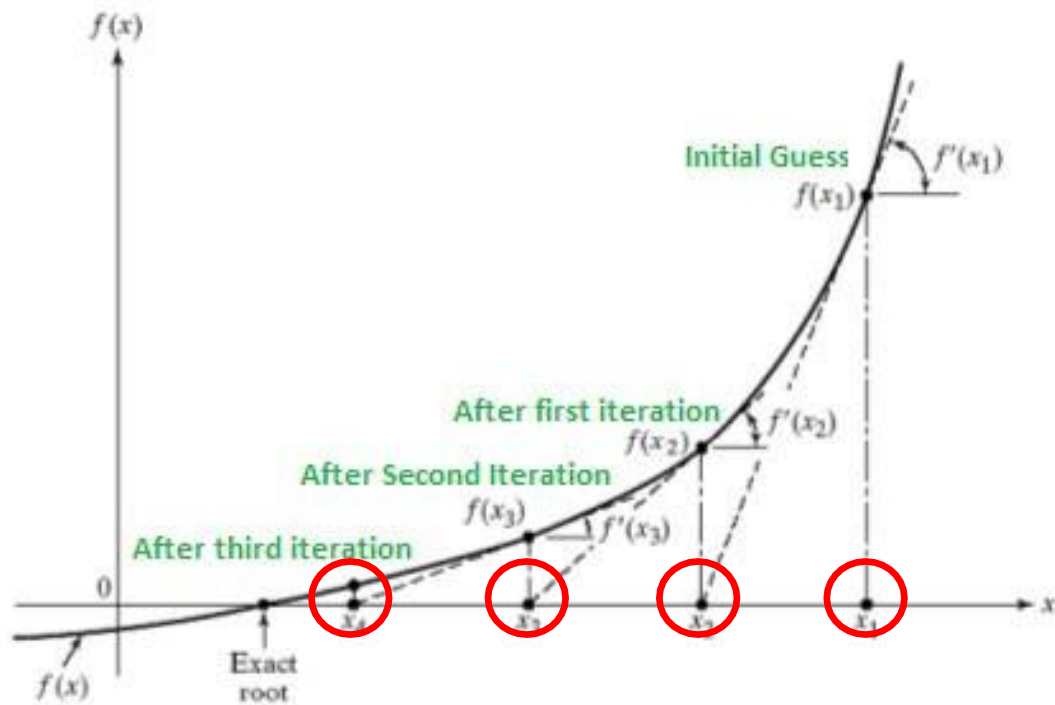
$$g - p(g)/p'(g)$$

is a better approximation; where  $p'$  is derivative of  $p$



# INTUITION - [LINK](#)

---



# NEWTON-RAPHSON ROOT FINDER

---

- ▶ Try to use this idea for **finding the square root of x**
  - ▶ Want to find  $r$  such that  $p(r) = 0$
  - ▶ To find the square root of  $k$ , find the root of  $p(x) = x^2 - k$
  - ▶ First derivative:  $2x$
- ▶ Newton-Raphson says given a guess  $g$  for root of  $k$ , a better guess is:

$$g - (g^2 - k)/2g$$



# NEWTON-RAPHSON ROOT FINDER

---

```
epsilon = 0.01
```

```
k = 24.0
```

$$f(x) = x^2 - 24$$

```
guess = k/2.0
```

```
num_guesses = 0
```

```
while abs(guess*guess - k) >= epsilon:
```

```
    num_guesses += 1
```

```
    guess = guess - ((guess**2) - k) / (2*guess)
```

$f(\text{guess})$

$f'(\text{guess})$

```
print('num_guesses = ' + str(num_guesses))
```

```
print('Square root of ' + str(k) + ' is about ' + str(guess))
```



# ITERATIVE ALGORITHMS

---

- ▶ Guess and check methods build on **reusing same code**
  - ▶ Use a looping construct
  - ▶ Generate guesses (important difference in algorithms)
  - ▶ Check and continue
- ▶ **Generating guesses**
  - ▶ Exhaustive enumeration
  - ▶ Approximation algorithm
  - ▶ Bisection search
  - ▶ Newton-Raphson (for root finding)





# DECOMPOSITION, ABSTRACTION, FUNCTIONs

# LEARNING to CREATE CODE

---

- ▶ So far have covered **basic language mechanisms** – primitives, complex expressions, branching, iteration
- ▶ In principle, **you know all you need to know** to accomplish anything that can be done by computation
- ▶ But in fact, we've taught you **nothing** about two of the most important concepts in programming...



# DECOMPOSITION and ABSTRACTION

---

## ► Decomposition

- How to divide a program into **self-contained parts** that can be combined to solve the current problem
  - Ideally **parts can be reused** by other programs
  - Self-contained means parts should **complete computation using only inputs provided** to them and “basic” operations

```
a = 3.14*2.2*2.2
```

```
pi = 3.14  
r = 2.2  
area = pi*r**2
```



# DECOMPOSITION and ABSTRACTION

---

## ▶ **Abstraction**

### ▶ How to ignore **unnecessary detail**

- ▶ Used to separate **what** something does, from **how** it actually does it
- ▶ Creating parts and abstracting away details allows us to write complex code while **suppressing details**, so that we are not overwhelmed by that complexity

```
# calculate the area of a circle
```



# DECOMPOSITION and ABSTRACTION

---

- ▶ Bake cake from a box
  - ▶ Prepare an empty container
  - ▶ Fetch boxes of dry ingredients
  - ▶ Pour ingredients into the container
  - ▶ Crack a few eggs into the container
  - ▶ Pour some milk into the container
  - ▶ Pre-heat the baker to 350F
  - ▶ Put the container into the baker
  - ▶ Set the timer to 5 minutes
  - ▶ Move the container out



# DECOMPOSITION and ABSTRACTION

---

- ▶ Bake cake from a box
  - ▶ 1) Mix dry ingredients
    - ▶ Prepare an empty container
    - ▶ Fetch boxes of dry ingredients
    - ▶ Pour ingredients into the container
  - ▶ 2) Add eggs and milk
    - ▶ Crack a few eggs into the container
    - ▶ Pour some milk into the container
  - ▶ 3) Bake at 350F for 5 minutes
    - ▶ Pre-heat the baker to 350F
    - ▶ Put the container into the baker
    - ▶ Set the timer to 5 minutes
    - ▶ Move the container out



# DECOMPOSITION and ABSTRACTION

---

- ▶ Bake cake from a box
  - ▶ 1) Mix dry ingredients
  - ▶ 2) Add eggs and milk
  - ▶ 3) Bake at 350F for 5 minutes



# BIG IDEA

Make code easy to

create

modify

maintain

understand



# FUNCTIONS

---

- ▶ Reusable pieces of code, called **functions** or **procedures**
- ▶ Capture steps of a computation so that we can use with any input
- ▶ A function is just some **code written in a special, reusable way**



# FUNCTIONS

---

- ▶ **Defining a function** tells Python some code now exists in memory
- ▶ Functions are only useful when they are **run** (“**called**” or “**invoked**”)
- ▶ You write a function once but can run it many times!



# FUNCTION CHARACTERISTICS

---

- ▶ Has a **name**
  - ▶ (think: variable bound to a function object)
- ▶ Has (formal) **parameters** (0 or more)
  - ▶ The inputs
- ▶ Has a **docstring** (optional but recommended)
  - ▶ A comment delineated by `"""` (triple quotes) that provides a **specification** for the function – contract relating output to input
- ▶ Has a **body**, a set of instructions to execute when function is called
- ▶ **Returns** something
  - ▶ Keyword `return`



# HOW to WRITE a FUNCTION

---

